PrepAI - Engineering Work Distribution

4-Member Balanced Team Allocation & Onboarding Reading Guide

Architecture: Vanilla JS + Node.js/Express + MongoDB + Gemini 1.5 Flash + Judge0 API | **Strategy:** Full-Stack Vertical Feature Slices | **Target:** 4 Software Engineers

---

1. Executive Allocation Strategy & Workload Balance

In modern agile engineering, **Vertical Slicing** assigns each engineer complete ownership of a functional domain from Frontend UI to Backend Controller, Database Schema, and Third-Party API integrations. This completely eliminates inter-member blocking (e.g. frontend engineers idling while backend finishes endpoints), drastically reduces Git merge collisions, and provides equal resume-caliber engineering impact for all 4 members.

Member	Primary Domain / Module	External Integrations	Frontend Scope	Backend & Database Scope	Weight
Member 1	**Authentication, Security & Aptitude Engine**	JWT, bcrypt, Nodemailer (OTP)	Login, Signup, OTP, Password Reset, Aptitude Test (6 pages)	authController, aptitudeController, User/Otp/Aptitude Models, Seeder	**25%**
Member 2	**Resume AI & Analytics Dashboard**	Google Gemini 1.5 Flash, Multer PDF	Landing Page, Profile Portfolio, Performance Dashboard (3 pages)	resumeController, resumeAnalysisController, userController, Multer	**25%**
Member 3	**Coding Assessment & Online Judge**	Judge0 RapidAPI, Ace Code IDE	Interactive Multi-Language IDE & Sandbox (1 heavy web app)	codingController, judge0Service, CodingQuestion/Result, Seeder	**25%**
Member 4	**AI Voice Interview & Admin Portal**	Gemini 1.5 Flash (Interview), Web Speech API	Voice Studio, Live Transcripts, Report Modal, Admin Panel (2 pages)	interviewController, adminController, Session/Question/Answer Models	**25%**

2. Universal Reading List (Required for ALL 4 Members Prior to Coding)

Before writing or editing code, every team member must read these shared architectural files to align on global conventions:

File Path	Key Takeaways & What to Understand
`README.md`	Complete architecture diagram, Mermaid flowchart, environment variable requirements, and run commands.
`backend/app.js & server.js`	Server bootstrap, Express middleware pipeline (Helmet security headers, CORS origins, rate limiters, route mounting).
`backend/.env.example`	Master list of environment variables: PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY, JUDGE0_API_URL/KEY.
`frontend/js/config.js`	Central API base URL resolution logic (switches automatically between localhost:5000 and production host).
`frontend/js/nav.js & css/style.css`	Shared navbar auth state detection, profile dropdown logic, and global CSS design tokens (colors, buttons, typography).

3. Member 1: Authentication, Security & Aptitude Engine

Core Responsibility: User registration/login lifecycle, email verification with OTP, password recovery, JWT session management, endpoint rate limiting, and the 15-question randomized aptitude test system.

Mandatory Reading List (in sequence):

File Path	Focus Areas & Architecture Understanding
`backend/models/User.js`	Read auth schema fields, password hashing pre-save hooks, bcrypt comparePassword method, JWT roles.
`backend/models/Otp.js`	Read OTP storage, 6-digit numeric generation, TTL index expiration, purpose enum ('verify_email', 'reset_password').
`backend/services/emailService.js`	Read Nodemailer SMTP configuration and HTML email layout used to send verification and reset OTP codes.
`backend/controllers/authController.js`	Examine register, login, verifyOTP, resendOTP, forgotPassword, resetPassword controller actions.
`backend/middleware/auth.js & rateLimiter.js`	Understand JWT Bearer token validation, attaching req.user, and window rate limiting rules.
`backend/models/AptitudeQuestion.js & AptitudeResult.js`	Examine question schema (category, options, correctOption, explanation) and result storage format.
`backend/seeder.js`	Read question bank seeding logic populating Quantitative, Logical, and Verbal questions.
`backend/controllers/aptitudeController.js`	Understand how /start randomizes 15 questions, strips correct answers, and how /submit calculates scores.
`frontend/pages/login.html & signup.html`	Review input element IDs, form validation triggers, error alert banners, and redirect flows.
`frontend/js/login.js, signup.js, verify-otp.js`	Examine token persistence in localStorage and auto-redirection logic.
`frontend/pages/aptitude.html & js/aptitude.js`	Study quiz interface: countdown timer, question palette navigator, option radio buttons, and score submit modal.

Files Owned: frontend/pages/login.html, signup.html, verify-otp.html, forgot-password.html, reset-password.html, aptitude.html | frontend/js/login.js, signup.js, verify-otp.js, forgot-password.js, reset-password.js, auth-ui.js, aptitude.js | frontend/css/auth.css, aptitude.css | backend/routes/authRoutes.js, aptitudeRoutes.js, healthRoutes.js | backend/controllers/authController.js, aptitudeController.js, healthController.js | backend/services/emailService.js, seeder.js | backend/models/User.js (Auth), Otp.js, AptitudeQuestion.js, AptitudeResult.js.

4. Member 2: Candidate Profile, Resume AI Parsing & Performance Dashboard

Core Responsibility: Candidate career profile management, Multer PDF upload pipeline, Google Gemini 1.5 Flash prompt engineering for ATS resume parsing, and the Central Performance Dashboard synthesizing cross-module scores.

Mandatory Reading List (in sequence):

File Path	Focus Areas & Architecture Understanding
`backend/models/User.js`	Read candidate profile attributes: headline, bio, skills array, education, experience, target job roles.
`backend/models/ResumeAnalysis.js`	Study schema for extracted resume metrics: ATS score, candidate name/phone, skills, domain fit, suggestions.
`backend/routes/resumeRoutes.js & controllers/resumeController.js`	Understand Multer file storage, 5MB file size limit, PDF mimetype validation, and disk path cleanup.
`backend/services/geminiService.js (Lines 1-250)`	CRITICAL: Study analyzeResumeWithGemini(), normalizeResumeData(), safeParseJSON(), and prompt schema.
`backend/utils/candidateNameValidator.js`	Inspect regex heuristics that distinguish real candidate names from false positives and timestamps.
`backend/controllers/resumeAnalysisController.js`	Examine pdf-parse integration converting PDF buffer to raw text before Gemini prompt dispatch.
`backend/routes/userRoutes.js & controllers/userController.js`	Study GET/PUT profile endpoints and candidate readiness calculation.
`frontend/index.html & css/landing.css`	Review landing page hero banner, features breakdown, and CTA navigation buttons.
`frontend/pages/profile.html & js/profile.js`	Study resume drag-and-drop zone, PDF upload status, parsed data cards, and profile editing form.
`frontend/pages/dashboard.html & js/dashboard.js`	CRITICAL: Study how Dashboard fetches Aptitude, Coding, and Interview scores to compute Job Readiness Score.

Files Owned: frontend/index.html, pages/profile.html, pages/dashboard.html | frontend/js/profile.js, dashboard.js, main.js | frontend/css/landing.css, profile.css, dashboard.css, style.css | backend/routes/resumeRoutes.js, userRoutes.js | backend/controllers/resumeController.js, resumeAnalysisController.js, userController.js | backend/services/geminiService.js (Resume section), candidateNameValidator.js | backend/models/ResumeAnalysis.js, User.js (Profile fields).

5. Member 3: Coding Assessment Engine & Online Judge (Judge0)

Core Responsibility: The browser IDE development environment (Ace Editor), cloud code compiler integration (Judge0 RapidAPI), DSA problem bank management, asynchronous execution polling, and sample/hidden test case evaluation.

Mandatory Reading List (in sequence):

File Path	Focus Areas & Architecture Understanding
`backend/models/CodingQuestion.js`	Study problem schema: title, difficulty (Easy/Medium/Hard), description markdown, constraints, sample and hidden test cases.
`backend/models/CodingResult.js`	Study submission model: user ref, code, language, status (Accepted, WA, TLE, Error), runtime, memory, tests passed.
`backend/codingSeeder.js`	Read the 30 DSA questions (Two Sum, Valid Parentheses, etc.), starter code templates, and hidden test cases.
`backend/services/judge0Service.js`	CRITICAL: Study Judge0 RapidAPI connection, language IDs (Python=71, JS=63, C++=54, Java=62), base64 encoding, and token polling loop.
`backend/controllers/codingController.js`	Understand /start (fetching 3 challenges by tier), /run (sample runner), and /submit (hidden grading and verdict generation).
`backend/routes/codingRoutes.js`	Review coding endpoints and rate limiter protection for expensive compiler execution calls.
`frontend/pages/coding.html`	Inspect layout: problem statement pane, test case drawer, language picker dropdown, Ace editor container, terminal output.
`frontend/js/coding.js`	CRITICAL: Study Ace editor setup, theme/mode switching, keyboard shortcuts, Run Code vs Submit Code event handling.
`frontend/css/coding.css`	Review styling for split-pane layout, code editor themes, execution spinner, and test case status pills (green/red).

Files Owned: frontend/pages/coding.html | frontend/js/coding.js | frontend/css/coding.css | backend/routes/codingRoutes.js | backend/controllers/codingController.js | backend/services/judge0Service.js | backend/codingSeeder.js | backend/models/CodingQuestion.js, CodingResult.js.

6. Member 4: AI Voice Mock Interview System & Admin Portal

Core Responsibility: The flagship voice interview experience: Web Speech API (STT & TTS), Gemini 1.5 Flash dynamic question generation, adaptive follow-ups, 5-dimension answer grading, final executive diagnostic report, and the Admin oversight dashboard.

Mandatory Reading List (in sequence):

File Path	Focus Areas & Architecture Understanding
`backend/models/InterviewSession.js`	Study interview session lifecycle: statuses (initialized, in-progress, completed), target job role, overall score, final summary.
`backend/models/InterviewQuestion.js`	Study dynamically generated questions: category (Technical, Problem Solving, Behavioral), difficulty, expected concepts.
`backend/models/InterviewAnswer.js`	Study answer scoring schema: transcript text, audio duration, 5 category scores (0-100), AI feedback, strengths, weaknesses.
`backend/services/geminiService.js (Lines 250-891)`	CRITICAL: Study generateInterviewQuestions(), evaluateInterviewAnswer(), generateFollowUpQuestion(), generateFinalInterviewReport().
`backend/controllers/interviewController.js`	Trace interview lifecycle endpoints: /start, /question, /answer, /end, and /report.
`backend/controllers/adminController.js & routes/adminRoutes.js`	Understand admin platform metrics: candidate counts, session logs, and system health summaries.
`frontend/pages/interview.html & css/interview.css`	Study interview studio UI: AI avatar state, question card, audio wave visualizer, transcript box, diagnostic report modal.
`frontend/js/interview.js (59 KB)`	CRITICAL: Deep dive into Web Speech API (SpeechRecognition for voice input, SpeechSynthesis for AI voice), silence timer, submission flow.
`frontend/pages/admin.html & js/admin.js & css/admin.css`	Study admin dashboard: candidate tables, session filters, and aggregate performance statistics.

Files Owned: frontend/pages/interview.html, admin.html | frontend/js/interview.js, admin.js | frontend/css/interview.css, admin.css | backend/routes/interviewRoutes.js, adminRoutes.js | backend/controllers/interviewController.js, adminController.js | backend/services/geminiService.js (Interview routines) | backend/models/InterviewSession.js, InterviewQuestion.js, InterviewAnswer.js.

7. Cross-Module API Contracts & Integration Points

To ensure seamless communication between modules without inter-member blocking, all members agree to the following API contracts:

Producer -> Consumer	Endpoint / Header Contract	Data Exchanged & Integration Purpose
Member 1 -> All	`Authorization: Bearer`	Decoded by auth middleware into req.user.id on all protected endpoints.
Member 2 -> Member 4	`GET /api/user/profile & /api/resume/analysis`	Member 4 reads candidate skills and target role to generate customized Gemini interview questions.
Member 1 -> Member 2	`GET /api/aptitude/result`	Returns aptitude test score, accuracy percentage, and category breakdown for the central dashboard.
Member 3 -> Member 2	`GET /api/coding/result`	Returns coding challenges solved, test cases passed, runtime performance, and language breakdown.
Member 4 -> Member 2	`GET /api/interview/report`	Returns 5-dimension interview score radar data (Technical, Communication, Problem Solving, Relevance, Structure).

All -> Member 2	`Aggregated Readiness Engine`	Dashboard synthesizes Aptitude (25%) + Coding (35%) + Interview (40%) into the final Job Readiness Score.

8. 4-Sprint Implementation Roadmap

Sprint & Phase	Key Deliverables by Member
Sprint 1: Foundations & Infrastructure	M1: MongoDB setup, User/Otp schemas, JWT auth flow, email OTP. M2: Landing page, Profile UI, Multer PDF upload pipeline. M3: Ace Code Editor setup, Judge0 API connection test. M4: Gemini 1.5 Flash API connection test, Web Speech API prototype.
Sprint 2: Core Assessment Engines	M1: Seed 30 aptitude questions, build timer & auto-grading logic. M2: Gemini prompt for resume ATS parsing, save ResumeAnalysis. M3: Implement sample test case runner & custom stdin runner. M4: Interview session lifecycle: dynamic question generation & voice capture.
Sprint 3: Advanced Scoring & UI Polish	M1: Password reset flow, rate limiting, anti-cheat test obfuscation. M2: Dashboard score aggregation cards & visual chart widgets. M3: Hidden test case submission evaluator & CodingResult persistence. M4: 5-dimension answer grading, final report generation, Admin dashboard.
Sprint 4: Integration & Production Deployment	All Members: End-to-end testing (Register -> Resume -> Aptitude -> Coding -> Interview -> Dashboard). Deploy backend on Render/Railway, MongoDB Atlas cluster, and frontend on Vercel.

9. Git Branching & Collaboration Guidelines

1. **Branch Convention:** Each member develops on an isolated feature branch: feature/auth-aptitude (M1), feature/resume-dashboard (M2), feature/coding-judge0 (M3), feature/interview-admin (M4).
2. **Protected Main:** Code merges into main only via Pull Requests reviewed by at least one other teammate.
3. **Environment Secrets:** Never commit .env. Keep .env.example updated with dummy keys.
4. **API Base URL:** Never hardcode http://localhost:5000 in scripts. Always use API_BASE_URL from config.js.